DESARROLLO DE APLICACIONES II

Desplegar y correr el Core

Módulo 9 · Core

20 de octubre de 2026

Contenido

Correrlo entero en tu máquina 3
La URL de Postgres se arregla sola 3
Dos opciones que cambian cuántos procesos hacen falta 3
Lo que el despliegue todavía no resuelve 4

El Core ya está desplegado. `https://core-uade-2026.duckdns.org`, con Postgres y RabbitMQ sobre AMQPS. Verificado el 17 de octubre.

- **Si sos de otro módulo y querés conectarte:** docs/integracion-aws.md. Es la guía válida y reemplaza los datos de conexión de cualquier documento anterior.
- **Si vas a operar el servidor:** deploy/README.md y el directorio `deploy/` (compose, systemd, backups, TLS, despliegue por OIDC).

Este documento es lo otro: **cómo correrlo en tu máquina** y qué hacen las dos opciones de configuración que afectan al despliegue.

Correrlo entero en tu máquina

```
docker compose up --build
```

Levanta Postgres, RabbitMQ y el Core con las mismas tres piezas que el servidor.

- API: `http://localhost:8000/docs`
- RabbitMQ: `http://localhost:15672` (guest/guest)

Sin Docker, los tres procesos a mano:

```
cd backend
.venv/bin/python -m app.seeds
.venv/bin/uvicorn app.main:app --reload
.venv/bin/python -m app.workers.inbox_worker   # en otra terminal
```

La URL de Postgres se arregla sola

Railway, Render, Heroku y Neon entregan la URL como `postgresql://` (y algunos todavía como `postgres://`). SQLAlchemy la lee como sincrónica, busca `psycopg2` —que no está instalado, porque el Core es async— y falla con `No module named 'psycopg2'`, que no dice nada del problema real.

El Core completa el driver solo, así que **podés pegar la variable tal cual** la dé el panel. Las tres formas funcionan:

```
postgres://...            → postgresql+asyncpg://...
postgresql://...          → postgresql+asyncpg://...
postgresql+asyncpg://...  → se deja como está
```

Dos opciones que cambian cuántos procesos hacen falta

Las dos están **apagadas por defecto**, y el despliegue de AWS no las usa: corre la API, el worker y el seed como contenedores separados, que es lo correcto cuando controlás el host.

`RUN_WORKER=true`
El servidor de la API también consume de `core.inbox` y `q.dlq` en su propio loop, en vez de necesitar `python -m app.workers.inbox_worker` aparte.

Sirve donde sólo podés pagar un proceso. El costo: los consumidores comparten el event loop con las peticiones HTTP, no se escalan por separado, y si uno se cae se cae todo. Para el volumen de este TP no se nota; con contenedores separados, no hace falta.

`SEED_ON_STARTUP=true`
El arranque corre el seed si hace falta. Es idempotente: no duplica nada ni rota secrets ya generados.

En este modo los secrets de máquina no se imprimen, a propósito. `stdout` son los logs de la plataforma y quedarían ahí para siempre, a la vista de cualquiera con acceso al panel — que es exactamente el hallazgo SEC-10 de la matriz de seguridad. En su lugar deja un aviso diciendo qué módulos tienen secret. Para obtener uno:

```
POST /api/v1/modules/{nombre}/rotate-secret
```

Devuelve uno nuevo, una sola vez, y se entrega **por un canal privado**.

Corriendo el seed a mano (`python -m app.seeds`) se siguen imprimiendo una vez, como siempre.

Lo que el despliegue todavía no resuelve

Hay un solo ambiente. El vhost `core` , compartido. El TP pide Test y Producción; hoy no están separados, y los eventos de prueba de un módulo llegan a los demás suscriptos. Hay que coordinarlos.

Publicando por AMQP, el Core no valida `sourceModule` . El 403 que devuelve `POST /events` es del camino HTTP. Por la cola, cualquiera puede publicar diciendo que es otro módulo. Con usuarios de RabbitMQ por módulo —que el despliegue ya tiene— falta el último paso: verificar el `user-id` del mensaje contra el sobre.